Ausgabe am **05.05.2022**
Abgabe bis zum **19.05.2023 13:30 Uhr**

Hochschule Fulda
University of Applied Sciences

Prof. Dr. Thomas Wiemann

Robotik
Übungsblatt 2

Sommersemester 2025

Ziel dieses Aufgabenzettels ist es, sich mit Transformationen und Odometrie auseinander zu setzen. Dazu werden wir zunächst das statische Modell aus der letzten Übung erweitern. Anschließend werden wir die in der Vorlesung vorgestellte Odometrie für einen Differenzialantrieb manuell berechnen und anschließend in ROS2-Code umsetzen. Am Ende der Übung werden wir in der Lage sein, dass Robotermodell mithilfe von Tastatureingaben in RViz zu bewegen.

Aufgabe A2.1 (Transformationen) (5P)

Gegeben sei die folgende Transformations-Matrix, die im $\mathbb{R}^2$ operiert.

$$T = \begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix}$$

(a) Erklären Sie, was eine Transformation ist.

(b) Welche Einträge der Matrix T sind Rotationskomponenten?

(c) Welche Einträge der Matrix T sind Translationskomponenten?

(d) Was muss mit einem Punkt im 2D Raum gemacht werden, damit er mit der Matrix T transformiert werden kann?

Aufgabe A2.2 (Joint State Publisher - Theorie) (20P)

Laden Sie den auf der Webseite der Veranstaltung bereitgestellten Workspace herunter. Er enthält ein Grundgerüst für die in dieser Übung zu bearbeitenden Programmieraufgaben. In der vorherigen Übung haben Sie ein einfaches, statisches URDF-Modell erstellt. Hier finden Sie im Package `fake_encoder` im Order `/urdf` ein Robotermodell für den Volksbot in der Datei `volksbot.urdf.xacro`. Dieses nutzt unter anderem `xacro`-Makros, um Variablen definieren zu können. Um die Syntax zu verstehen, empfehlen wir folgendes ROS2-Tutorial durchzuarbeiten `https://docs.ros.org/en/humble/Tutorials/Intermediate/URDF/Using-Xacro-to-Clean-Up-a-URDF-File.html`. Im Gegensatz zum statischen Modell aus der vergangenen Woche werden hier zur Verbindung der Räder keine festen Joints vom Typ `fixed` modelliert, sondern `continuous` Joints. Diese repräsentieren eine Rotation um eine vorgegebene Achse. Damit diese Rotationen definiert sind, müssen die entsprechenden Joint-States über einen sogenannten Joint-State-Publisher auf dem Topic `joint_states` veröffentlicht werden. Erweitern Sie dazu die vorgegebene Node `FakeEncoder` um einen Publisher für Messages vom `sensor_msgs::msg::JointState`. Diese sinds wie folgt definiert:

```
std_msgs/Header header
string[] name
```

```
float64[] position
float64[] velocity
float64[] effort
```

Der Header enthält im Feld `header.stamp` einen Zeitstempel, der angibt wann die Nachricht erzeugt wurde. Nutzen Sie die Node-Funktion `now()`, um Sie mit dem aktuellen Zeitpunkt zu füllen. Das Array `name` enthält die Namen aller Joints, die geupdatet werden sollen. Dazu enthält Ihre Klasse bereits ein passendes Feld `joint_names_`, das die in der URDF-Datei genutzten Namen enthält. Wichtig ist das Feld `position`, dass die aktuelle Drehung in Radians enthält.

Erweitern Sie die Klasse `FakeEncoder` um einen Timer, der mit einer Periode von 100 ms den Drehwinkel 1 Grad erhöht. *Achtung*: Denken Sie daran, die Grad-Werte ins Bogenmaß zu konvertieren und im Intervall $[0, 2\pi]$ zu halten. Für die Implementierung des Timer können Sie den Code aus dem Chatter-Beispiel der Tutorials der letzten Woche verwenden. Wenn Sie alle Joint-States korrekt publishen, können Sie das Robot-Modell in RViz visualisieren. Starten Sie dazu das Launch-File `fake_encoder.launch.py` aus dem Package. Anschließend können Sie in RViz ein Robotermodell hinzufügen:

```
ros2 run rviz2 rviz2
```

Dann mittels "Add" ein `RobotModel` hinzufügen. Setzen im Robot-Model-Display den Wert "Description Source" auf `Topic` und das "Description Topic" auf `robot_description`. Bei korrekter Implementierung sollten Sie ein Modell des Volksbots sehen, dessen Räder sich langsam gleichmäßig drehen.

Aufgabe A2.3 (Absolutgeber - Theorie) (15P)

Gegeben Sei ein Roboter mit zwei Motoren, jeweils ein Motor pro Seite: M_l und M_r. Jeder dieser beiden Motoren besitzt einen Absolutgeber, welche Signale nach dem Schema aus Abbildung 1 erzeugt. Der linke Motor M_l erzeugt bei einer Geradeausfahrt Daten gegen den Uhrzeigersinn. Wegen der Bauart des Roboters, mussten die Motoren entgegengesetzt zueinander montiert werden.

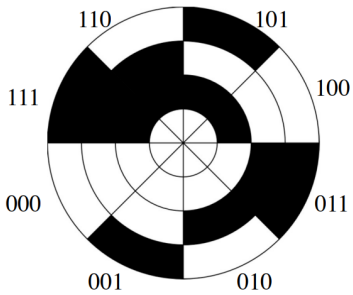

Abbildung 1: Absolutgeber

Während der Roboter fährt, zeichnen Sie die folgenden Daten auf:

T	0	1	2	3	4	5	6	7	8	9
M_l	101	110	111	000	001	010	011	100	101	110
M_r	000	000	111	111	110	110	101	101	100	100

(a) Beschreiben Sie, wie der Roboter vermutlich gefahren ist.

(b) Der Roboter hat Reifen mit einem Durchmesser von 0.3m. Berechnen Sie, wie weit (in m) jeder dieser Reifen zu jedem Zeitpunkt an Strecke zurückgelegt hat.

T	0	1	2	3	4	5	6	7	8	9
D_l	0	...	...	...	...	...	...	...	...	...
D_r	0	...	...	...	...	...	...	...	...	...

(c) Erklären Sie welche Probleme mit der gegebenen Kodierung aus Abbildung 1 auftreten könnten. Geben Sie dazu auch ein Beispiel an. Wie könnte man die Kodierung verbessern?

Aufgabe B2.1 (Differenzialantrieb) (20P)

Gegeben sei ein Roboter mit Differenzialantrieb an Pose $(0\,\text{cm}, 0\,\text{cm}, 0°)$ zum Zeitpunkt $t = 0$.

Die Kinematik für einen Differenzialantrieb sei wie folgt gegeben:

$$\Delta d = \frac{(s_l + s_r)}{2}$$
$$\Delta\theta = \frac{(s_r - s_l)}{b}$$

Δd entspricht dabei der gefahrenden Strecke des Roboters, $\Delta\theta$ der Winkeländerung. Die Räder der linken/rechten Seite haben einen Abstand von $b = 30\,\text{cm}$.

(a) Zur Zeit $t = 1$ wird gemessen, dass sich das rechte Rad $s_r = 6\,\text{cm}$ bewegt hat, das linke $s_l = 10\,\text{cm}$. An welcher Pose befindet sich der Roboter nun? Für $t = 2$ liefert die Messung: Links $5\,\text{cm}$, rechts $7\,\text{cm}$. Wo steht der Roboter nun?

(b) Sie stellen fest, dass der Roboter nicht genau arbeitet. Bei jeder Rotation und Translation ist die resultierende Pose geringfügig ungenau. Argumentieren Sie welcher Fehler (Rotation oder Translation) sich langfristig stärker auswirkt.

Aufgabe B2.2 (Lokaliserungsqualität) (20P)

Auf dem RosBot2 gibt es die Topics `/imu_broadcaster/imu` auf denen Positonsschätzungen der internen IMU das Roboters gepublisht werden. Zusätzlich gibt es das Topic `/rosbot_base_controller/odom`, das eine gefilterte Schätzung der Roboterposition ausgibt (mehr dazu in den kommenden Vorlesungen). Schreiben Sie eine ROS2-Node, die sich zu diesen Topics verbindet und jeweils die geschätzten x- und y-Positionen in ein Text-File ausgibt. Schauen Sie sich dazu die Definitionen der `/nav_msgs/msg/Odometry` und `/sensor_msgs/msg/Imu` an. Schreiben Sie Ihre berechneten x und y-Koordinaten Zeilenweise in separate Textdateien, z.B. `odom.txt` und `imu.txt`. Dazu können Sie in C++ einen `std::ofstream` verwenden. Eine Rahmenklasse dazu wird Ihnen in Moodle zur Verfügung gestellt. Steuern Sie den Roboter mit einer Teleop-Node entlag eines definierten Pfades und loggen Sie dabei die Positionen. Plotten Sie die Trajektorien geeignet z.B. mit `gnuplot`:

```
% gnuplot
% plot 'odom.txt' u 1:2,'imu.txt' u 1:2
```

Fahren Sie eine bekannte Strecke und schätzen Sie den Fehler ab. Bei welchen Operationen wird der Fehler besonders hoch?

Abgabe

Geben Sie Ihre Lösungen bis zum 19.05.2025, 13:30 über die entsprechende Moodle-Aufgabe ab.